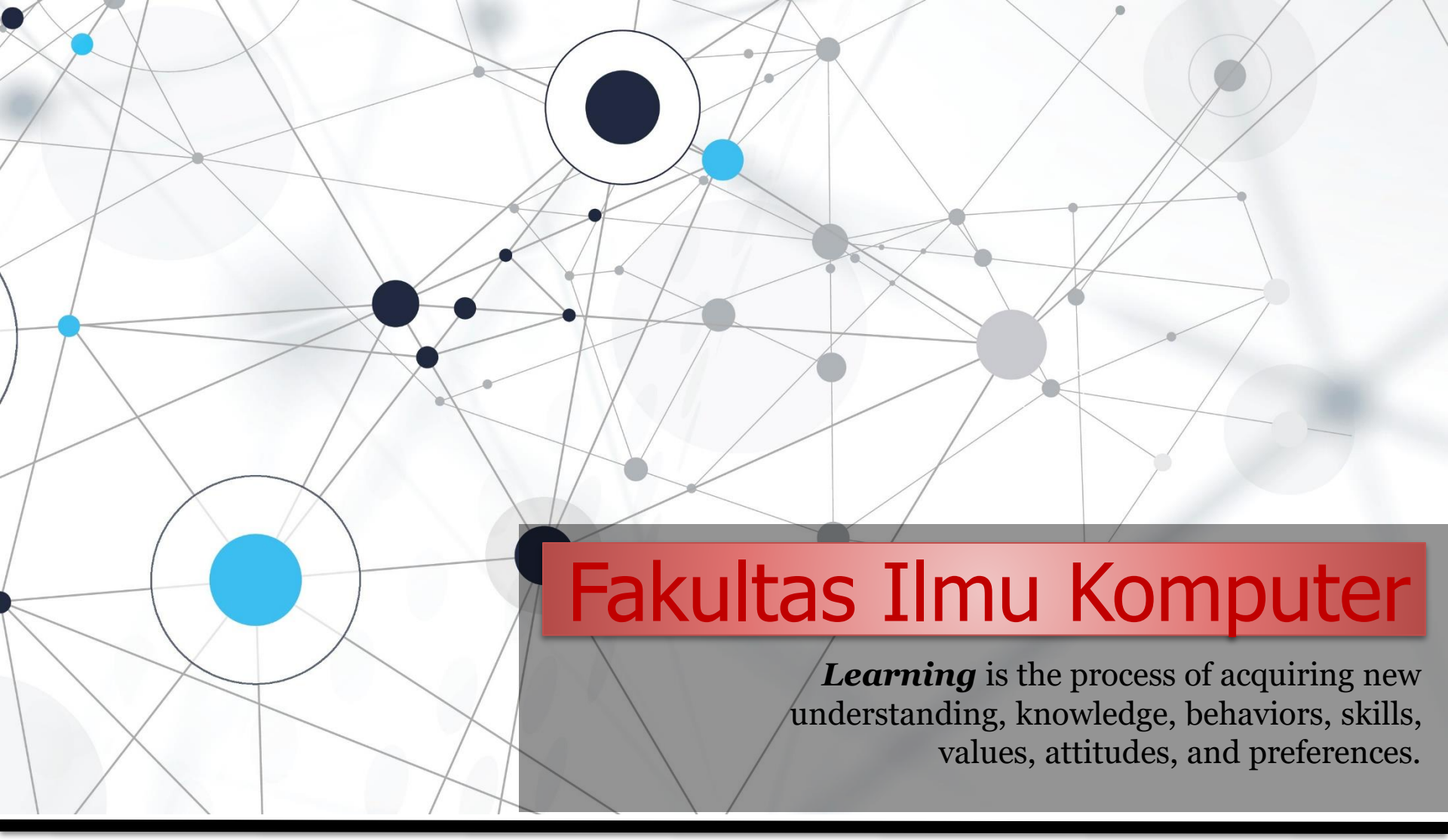
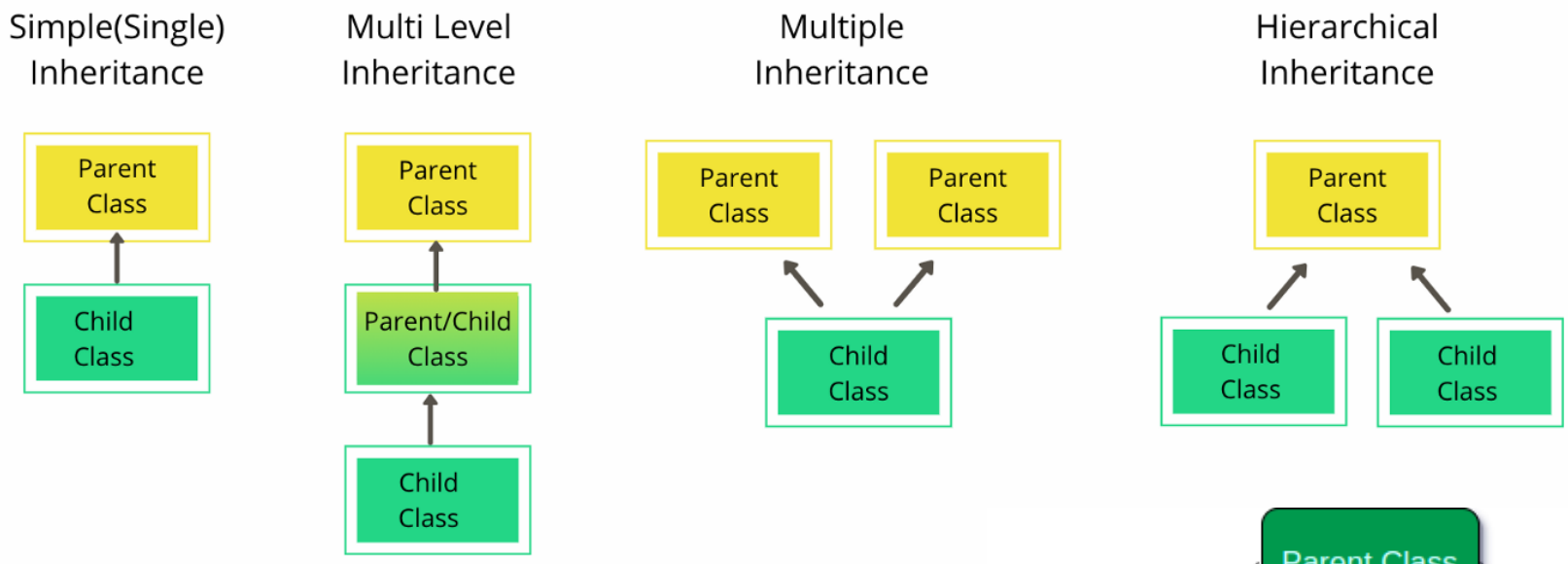


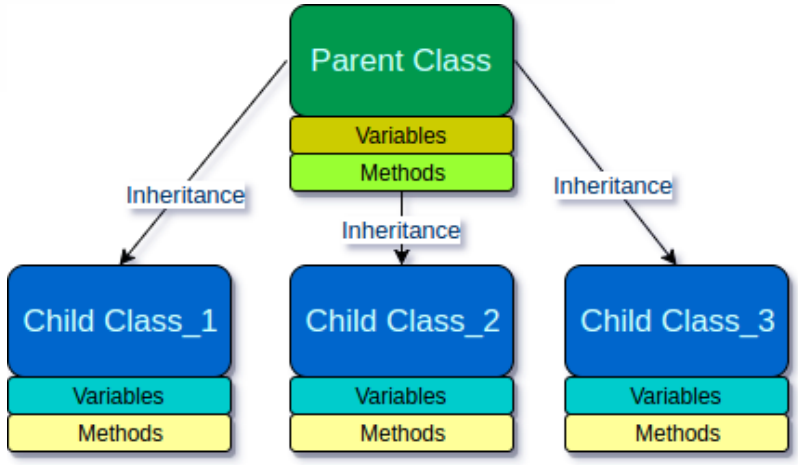
OOP *in* Python – Polymorphism

Welcome to *the* Object Oriented Programming *course*

Parent Class *vs* Child Class

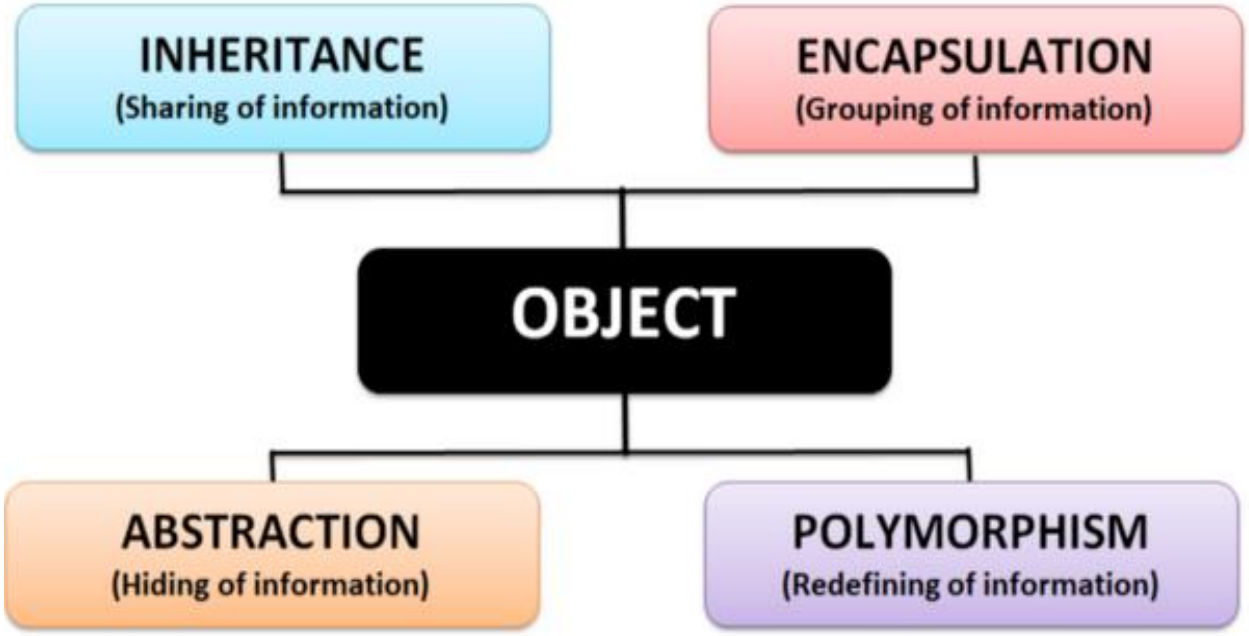
Parent class is the class being inherited from, also called base class. Child class is the class that inherits from another class, also called derived class.





1 – What *is* Polymorphism?

Understanding the Concept *of* OOP



- ✓ Object Oriented Programming (OOP) atau pemrograman berorientasi objek adalah salah satu pendekatan paling efektif dalam menulis kode suatu program/software.
- ✓ Pada OOP, kita menulis class yang merepresentasikan suatu situasi seperti pada keadaan sebenarnya dan kita dapat membuat suatu objek berdasarkan class tersebut.

The four pillars of object-oriented programming in Python are: Abstraction; Encapsulation; Inheritance; Polymorphism.

What *is* Polymorphism?

- **Polymorphism** adalah **kemampuan untuk mengambil bentuk yang berbeda**. Polymorphism dalam Python memungkinkan kita untuk **mendefinisikan metode pada child class dengan menggunakan nama yang sama seperti pada parent class**.

- Polymorphism defines the **ability to take different forms**. Polymorphism in Python allows us to define methods in the child class with the same name as defined in their parent class.

- Polymorphism is the principle that one kind of thing **can take a variety of forms**. In the context of programming, this means that a single entity in a programming language can behave in multiple ways depending on the context.

- Polymorphism refers to **having multiple forms**. Polymorphism is a programming term that refers to the use of the same function name, but with different signatures, for multiple types.

Polymorphism digabung dari dua kata:

Poly = banyak

Morphism = bentuk (*form*)

Fungsi yang sama dapat digunakan pada *tipe* yang berbeda.

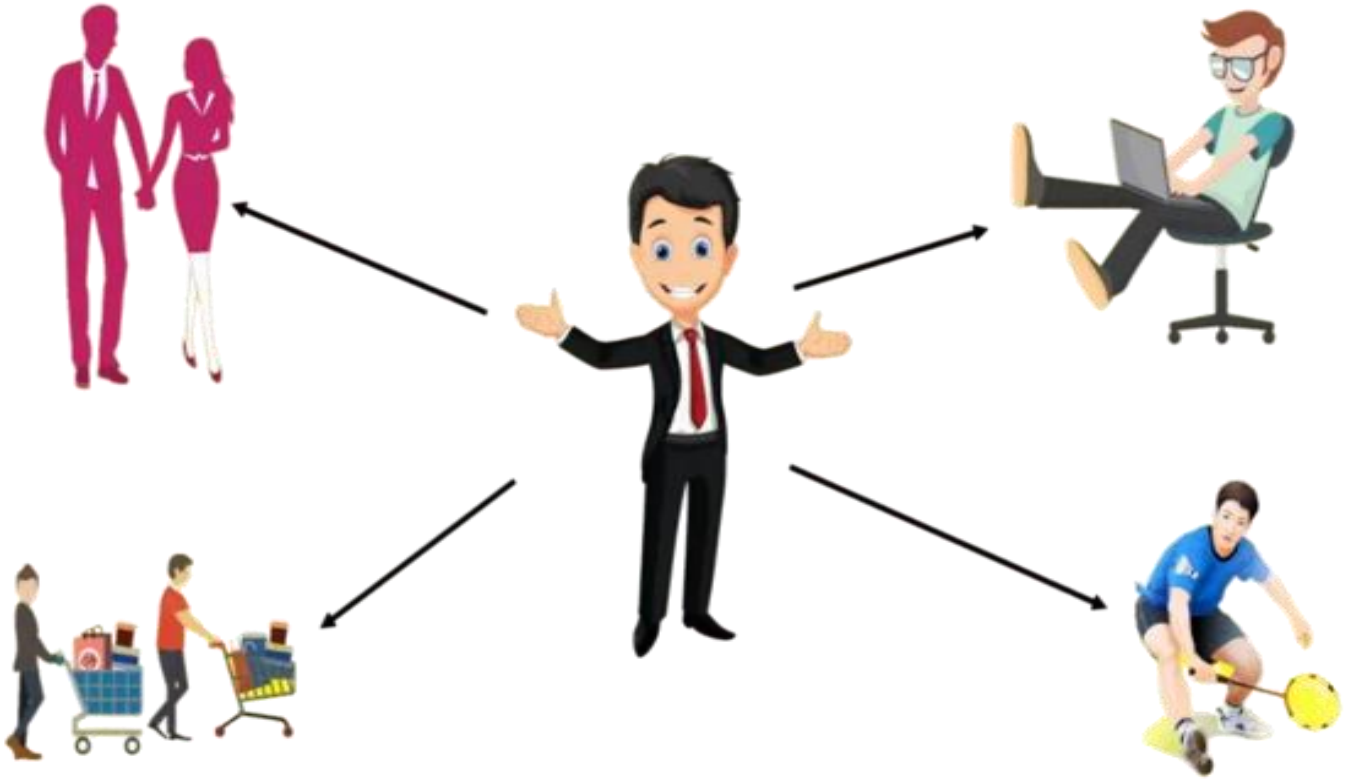
Dapatkah kalian memberikan contoh satu jenis object/benda/orang yang dapat mengambil berbagai bentuk yang berbeda?

OOP Ilustrasi Polymorphism

Polymorphism adalah kemampuan untuk mengambil bentuk yang berbeda.

OOP Ilustrasi Polymorphism

Polymorphism adalah kemampuan untuk mengambil bentuk yang berbeda.



You can take different forms as per the situation.

OOP Ilustrasi Polymorphism

Polymorphism adalah kemampuan untuk mengambil bentuk yang berbeda.

Employee Role

Wife Role

Tennis Player Role

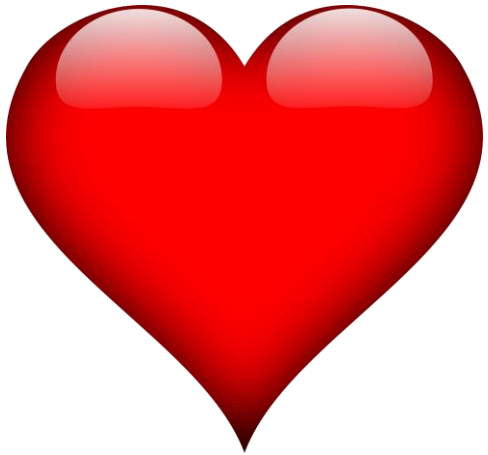
You can take different forms as per the situation.

OOP Ilustrasi Polymorphism

Polymorphism adalah kemampuan untuk mengambil bentuk yang berbeda.

OOP Ilustrasi Polymorphism

Polymorphism adalah bukan sebuah kemampuan untuk mempunyai banyak pasangan.



Naruto kage bunshin no jutsu


Sage Art: Super Tailed Beast
Rasen-Shuriken!

What *is* Polymorphism?

Polymorphism can be defined as a **condition that occurs in many different forms**. It is a concept in Python programming wherein **an object defined in Python can be used in different ways**. It allows the programmer to define multiple methods in a derived class, and it has the same name as present in the parent class. Such scenarios support **method overloading** in Python.

What *is* Polymorphism?

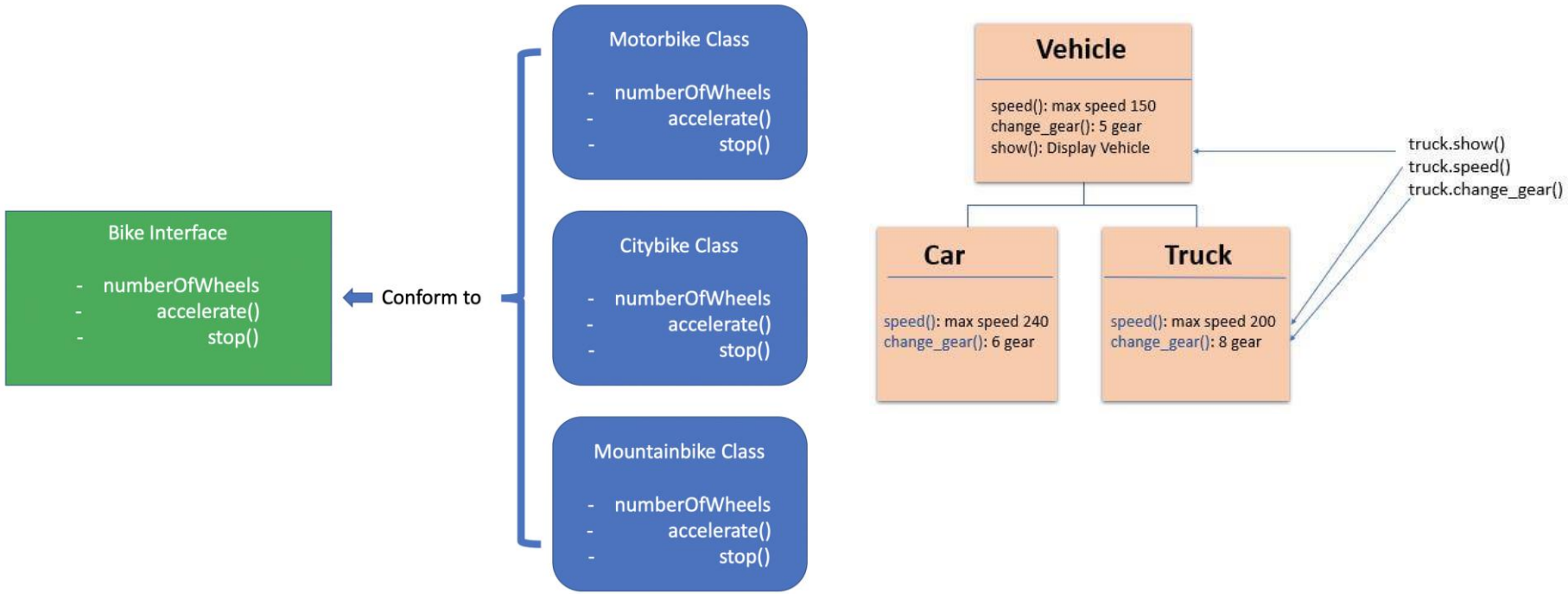
Polymorphism adalah sebuah cara atau *ability* untuk mengambil bentuk yang berbeda. Dalam context ini, *polymorphism* memungkinkan kita untuk mendefinisikan *method* pada **child class** dengan **menggunakan nama yang sama** seperti pada **parent class**.

What is the relation between parent and child class?

A parent class contains the child class. Where as a derived class inherits from a base class. They are similar because the child (*or derived*) can access the parents (*or base*) properties and methods (*where allowed*). They are different because we can refer to a property of the child class in the form of Parent.

Polymorphism *in* Python

Polymorphism dalam Python memungkinkan kita untuk mendefinisikan metode pada *child class* dengan menggunakan nama yang sama seperti pada *parent class*.



Class child mewarisi seluruh method dari *parent class*. Tapi, ada beberapa kasus di mana **metode tersebut tidak cocok** dengan *child class*. Oleh karena itu, kita harus mengimplementasikan (*re-used*) kembali metode yang pada *child class* yang dinamakan **Method Overriding**.

[1] Polymorphism *with* Class

```python
# create class 1
class Cat:
    # constructor (initialize instance variable)
    def __init__(self, nama, umur):
        self.nama = nama
        self.umur = umur

    def speak(self):
        print("Cat speaks Meow Meow Meow.")

# create class 2
class Dog:
    # constructor (initialize instance variable)
    def __init__(self, nama, umur):
        self.nama = nama
        self.umur = umur

    def speak(self):
        print("Dog speaks Gukk Gukk Gukk.")

# object of Cat created
kucing1 = Cat("Catty", 5)

# object of Dog created
anjing1 = Dog("Doggy", 4)

# memanggil metode tanpa mengabaikan objek
kucing1.speak()
anjing1.speak()

# memanggil metode dengan mengabaikan objek
for hewan in (kucing1, anjing1):
    hewan.speak()
```

1

2

- We can use the concept of polymorphism when creating methods for a class. Python allows different classes to use methods with the same name. We can then call the method by ignoring the object we are using.
- Kita dapat menggunakan konsep polymorphism ketika membuat metode sebuah Class.
- Bahasa pemrograman Python memungkinkan sebuah class yang berbeda untuk menggunakan metode dengan nama yang sama.
- Kemudian kita dapat memanggil metode tersebut dengan mengabaikan objek yang sedang kita gunakan.

[2] Polymorphism *with* Inheritance

```python
# create parent class
class Bird:
    def intro(self):
        print("There are several different types of bird species.")

    def fly(self):
        print("Almost all birds can fly, but there are some that cannot.")

# create child class 1
class Eagle(Bird):

    def fly(self):
        print("Yes, an eagle can fly.")

# create child class 1
class Weka(Bird):

    def fly(self):
        print("No, an Weka can't fly.")

# # object of parent class & child class created
obj_burung = Bird()
obj_elang = Eagle()
obj_weka = Weka()

# user-defined function is called from obj_burung (parent class)
obj_burung.intro()
obj_burung.fly()

# user-defined function is called from obj_elang (child class)
obj_elang.intro()
obj_elang.fly()

# user-defined function is called from obj_weka (child class)
obj_weka.intro()
obj_weka.fly()
```

1 2 3

- Polymorphism in object-oriented programming with Python defines a *child class* that has the same method names as the *parent class*. In inheritance, the **child class** inherits methods from the **parent class**. In addition, **it is possible to modify methods** in the child class that have been inherited from the parent class.

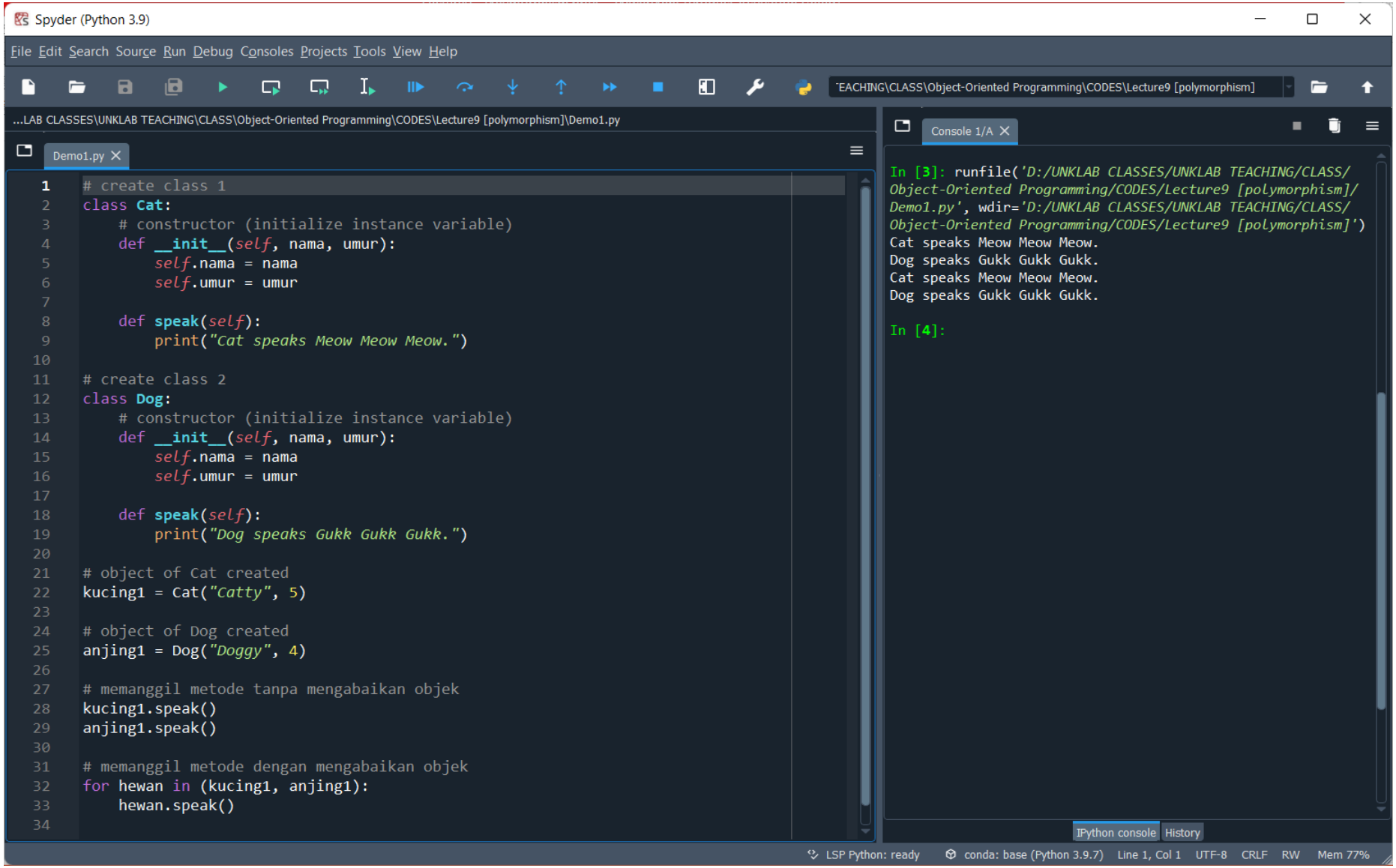
- Polymorphism pada pemrograman berorientasi objek dengan Python mendefinisikan *child class* yang memiliki kesamaan nama metode pada *parent class*. Pada inheritance, *child class* mewarisi metode dari *parent class*.

- Selain itu, **memungkinkan untuk memodifikasi metode** di dalam child class yang telah diwarisi dari parent class.

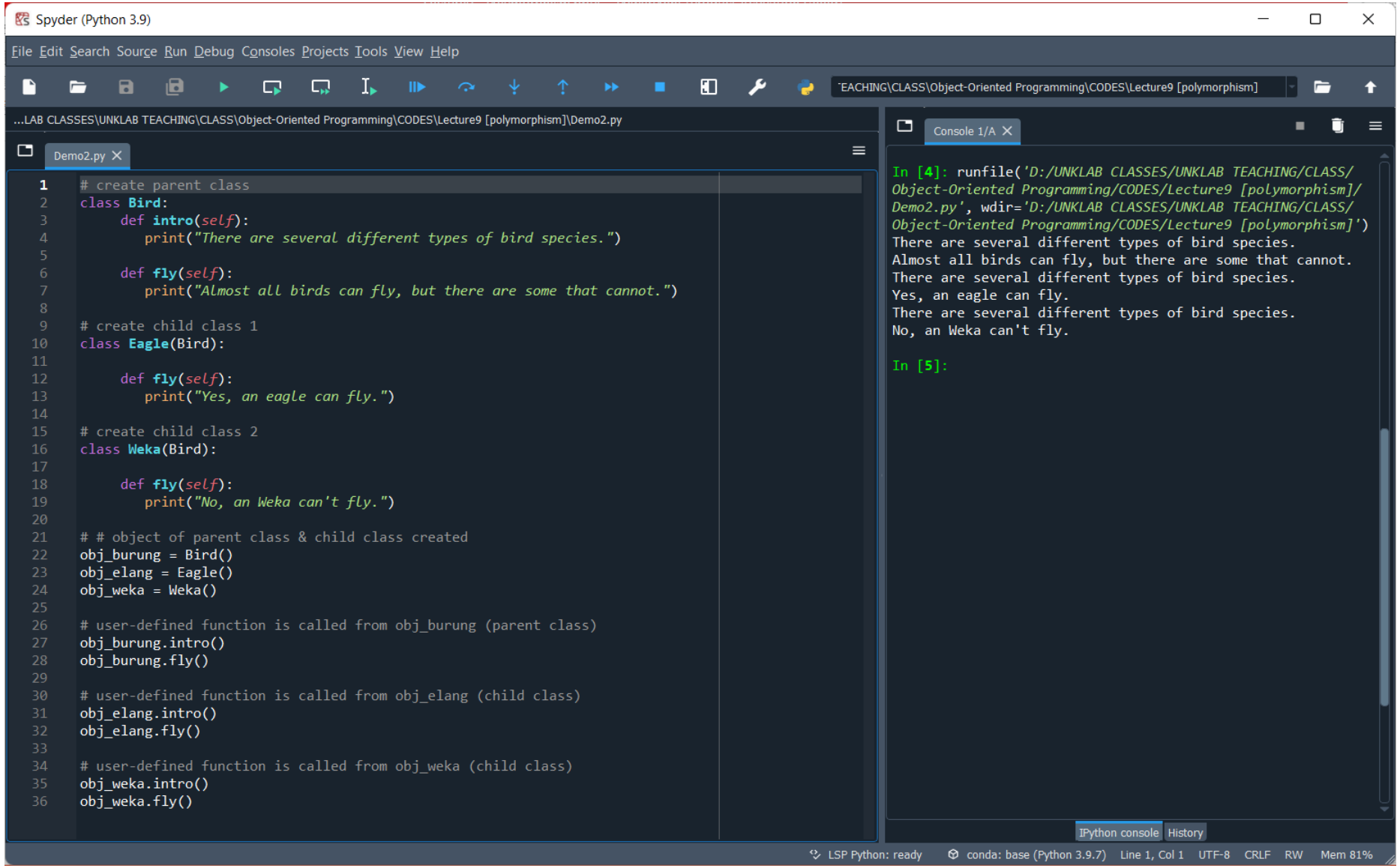
Polymorphism is a concept where objects of different classes can have different behaviors when calling the same method.

Exercise *for* Students

Lab Exercise #1



Lab Exercise #2



Discuss, answer questions and 6 students will be asked to participate in the class.

1) How is the concept of polymorphism applied in Demo2.py code? (Bagaimana konsep polymorphism diterapkan dalam kode Demo2.py di atas?)

2) From three classes in Demo2.py file, choose which one is the *parent class* and which one is the *child class*? Why? (Dari tiga class yang ada pada file Demo2.py, pilihlah kelas mana yang merupakan *kelas induk* dan kelas mana yang merupakan *kelas turunan*? Mengapa?)

3) What is the main difference between the **fly() method** in the parent class Bird and the child class Eagle? (Apa perbedaan utama antara *fly() method* dalam parent class Bird dan child class Eagle?)

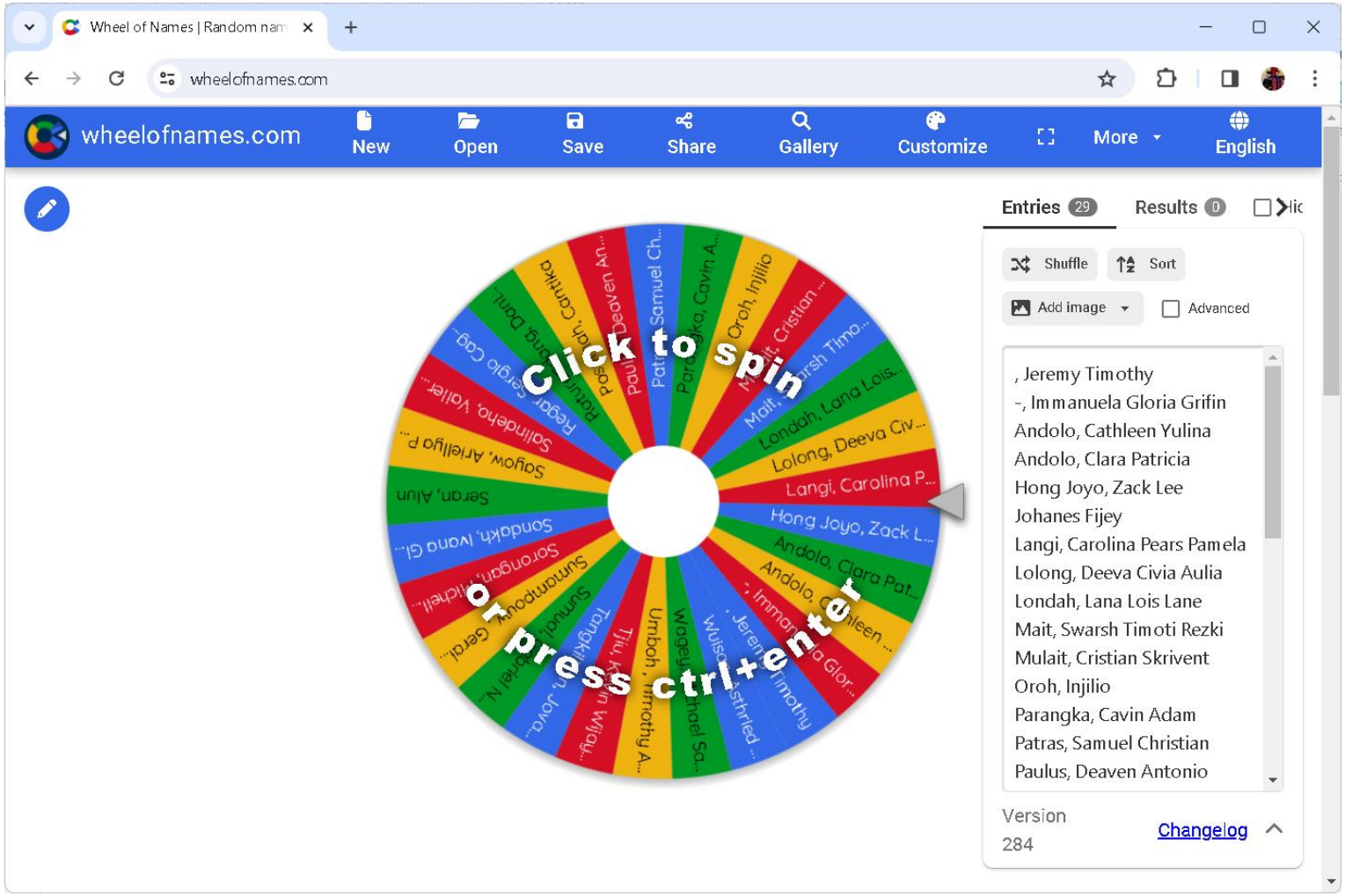
4) Why does the **fly() method** in the child class Weka have different behavior from the **fly() method** in the parent class Bird? (Mengapa *fly() method* dalam *child class Weka* memiliki perilaku yang berbeda dari *fly() method* dalam *parent class Bird*?)

5) Why does the child class Eagle override the *fly() method* of the parent class Bird? What are its benefits in the context of polymorphism? (Mengapa child class Eagle menimpa metode fly dari parent class Bird? Apa manfaatnya dalam konteks polymorphism?)

6) What happens when *intro() method* is called from objects of different classes, such as *obj_bird*, *obj_eagle*, and *obj_weka*? (Apa yang terjadi ketika *intro() method* dipanggil dari objek-objek kelas yang berbeda, seperti *obj_burung*, *obj_elang*, dan *obj_weka*?)

URL:

https://wheelofnames.com/



END PRESENTATION

Thank you for your attention

Instructor: S – W – T